

CS 170 Midterm 1

Write in the following boxes clearly and then double check.

Name :

SID :

Exam Room :

- ☐ Dwinelle 155
- ☐ VLSB 2050
- ☐ Other (Specify):

Name of student to your left :

Name of student to your right :

- **The exam will last 110 minutes.**
- The exam has 8 questions with a total of 170 points. You may be eligible to receive partial credit for your proof even if your algorithm is only partially correct or inefficient.
- Only your writing inside the answer boxes will be graded. **Anything outside the boxes will not be graded.** The last page is provided to you as a blank scratch page.
- Answer all questions. Read them carefully first. Not all parts of a problem are weighted equally.
- The problems may **not** necessarily follow the order of increasing difficulty.
- The points assigned to each problem are by no means an indication of the problem's difficulty.
- The boxes assigned to each problem are by no means an indication of the problem's difficulty.
- **If a question asks you to provide an algorithm, you do not need to provide a proof of correctness or runtime analysis unless explicitly stated.**
- Unless the problem states otherwise, you should assume constant-time arithmetic on real numbers. Unless the problem states otherwise, you should assume that graphs are simple.
- If you use any algorithm from lecture and textbook as a black box, you can rely on the correctness and time/space complexity of the quoted algorithm. If you modify an algorithm from textbook or lecture, you must explain the modifications precisely and clearly, and if asked for a proof of correctness, give one from scratch or give a modified version of the textbook proof of correctness.
- Unless stated, only deterministic (non-random) algorithms will receive points.
- Assume the subparts of each question are **independent** unless otherwise stated.
- **Please write your SID on the top of each page; you will get 1 point for doing so.**
- **For multiple choice questions, please fill in the bubbles fully.**
- Good luck!

1 Asymptotics warmup (13 points)

- (a) (5 points) For each pair of functions f and g , specify whether $f = O(g)$, $g = O(f)$, or both.

f	g	$f = O(g)$	$g = O(f)$	Both
n^2	$\log_2(n^{99})$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$n^{100} + 99^n$	$n^{99} + 100^n$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$n^{\log_2 \log_2 n}$	$(\log_2 n)^{\log_2 n}$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$n^{\log_2 n}$	$(\log_2 n)^n$	$\bigcirc$	$\bigcirc$	$\bigcirc$
$2^{\sqrt{\log_2 n}}$	n	$\bigcirc$	$\bigcirc$	$\bigcirc$

Solution:

- (a) $g = O(f)$ only. Taking their ratio and using L'hôpital's rule, notice that

$$\lim_{n \rightarrow \infty} \frac{f}{g} = \lim_{n \rightarrow \infty} \frac{n^2}{\log_2 n^{99}} = \lim_{n \rightarrow \infty} \frac{2n}{\frac{99 \ln 2}{n}} = \lim_{n \rightarrow \infty} \frac{2n^2}{99 \ln 2} = \infty.$$

In general, $\log n = O(n^c)$ for any positive constant $c > 0$.

- (b) $f = O(g)$ only. Taking their ratio, we see that

$$\lim_{n \rightarrow \infty} \frac{f}{g} = \lim_{n \rightarrow \infty} \frac{n^{100} + 99^n}{n^{99} + 100^n} = \lim_{n \rightarrow \infty} \frac{\frac{n^{100}}{99^n} + 1}{\frac{n^{99}}{99^n} + \left(\frac{100}{99}\right)^n} = \frac{0 + 1}{0 + \infty} = 0.$$

- (c) Both. By applying the rules of logarithms, one can see that, in fact, $f = g$.

- (d) $f = O(g)$ only. Notice that $f = 2^{(\log n)^2}$, whereas $g = 2^{n \log_2 \log_2 n}$. Taking their ratio, we see that

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f}{g} &= \lim_{n \rightarrow \infty} \frac{2^{(\log_2 n)^2}}{2^{n \log_2 \log_2 n}} = \lim_{n \rightarrow \infty} 2^{(\log_2 n)^2 - n \log_2 \log_2 n} \\ &= 2^{\lim_{n \rightarrow \infty} n \cdot \left(\frac{(\log_2 n)^2}{n} - \log_2 \log_2 n \right)} = 2^{\infty \cdot (0 - \infty)} = 2^{-\infty} = 0. \end{aligned}$$

- (e) $f = O(g)$ only. Taking their ratio, we see that

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{f}{g} &= \lim_{n \rightarrow \infty} \frac{2^{\sqrt{\log_2 n}}}{2^{\log_2 n}} = \lim_{n \rightarrow \infty} 2^{\sqrt{\log_2 n} - \log_2 n} \\ &= \lim_{n \rightarrow \infty} 2^{\sqrt{\log_2 n} (1 - \sqrt{\log_2 n})} \\ &= 2^{\lim_{n \rightarrow \infty} \sqrt{\log_2 n} (1 - \sqrt{\log_2 n})} = 2^{\infty (1 - \infty)} = 2^{-\infty} = 0. \end{aligned}$$

For each of the following recurrences, state a closed form for $T(n)$. For each of the recurrence relations, $T(1) = T(0) = 1$. **A proof is not required for any of the recurrence relations.**

(b) (2 points) $T(n) = 10T(n/10) + O(n)$

Solution: $T(n) = O(n \log n)$. This follows from a direct application of the Master Theorem.

(c) (2 points) $T(n) = T(n-1) + O(n \log n)$

Solution: $T(n) = O(n^2 \log n)$. By unravelling the recurrence, we see that $T(n) = \sum_{i=1}^n O(i \log i) = O(\sum_{i=1}^n i \log i)$. Notice that $\sum_{i=1}^n i \log i \geq n/2 \cdot (n/2) \log(n/2) = (n^2/4) \log(n/2)$ and $\sum_{i=1}^n i \log i \leq n \cdot n \log n = n^2 \log n$. Thus, we conclude that $T(n) = O(n^2 \log n)$.

(d) (2 points) $T(n) = 2T(\sqrt{n}) + O(n)$

Solution: $T(n) = O(n)$. By unravelling the recurrence, we see that $T(n) = O(\sum_{i=0}^{\log \log n} 2^i n^{1/2^i}) = -O(n + \sum_{i=1}^{\log \log n} 2^i n^{1/2^i}) = O(n + \log \log n \cdot \log n \cdot \sqrt{n}) = O(n)$.

(e) (2 points) $T(n) = 2T(\sqrt{n}) + O(\log n)$

Solution: $T(n) = O(\log n \log \log n)$. Using the substitution $S(m) \triangleq T(2^m)$, the recurrence translates to the recurrence $S(m) = 2S(m/2) + O(m)$. By the Master Theorem, we see that $S(m) = O(m \log m)$. Plugging this back in, we conclude that $T(n) = O(\log n \log \log n)$.

2 Miscellany (22 points)

- (a) (2 points) If two vertices u and v belong to the same strongly connected component, then the intervals $[\text{pre}(u), \text{post}(u)]$ and $[\text{pre}(v), \text{post}(v)]$ must overlap.

☐ True ☐ False

Solution: True. Every vertex in an SCC is reachable from any other vertex within the SCC, so there is a path from u to v and from v to u . When DFS visits one of these vertices, it will visit the other vertex before it finishes exploring the first vertex. As a result, their DFS intervals overlap.

- (b) (2 points) In a graph with n vertices and e edges, suppose Kosaraju's algorithm finds k SCCs. Up to how many vertices can be contained inside the largest SCC? Give your answer in terms of n , e , and/or k .

Solution: $n - k + 1$. Since there are k SCCs, at least 1 vertex must be in each SCC. To maximize the size of 1 SCC, the other remaining $(k - 1)$ SCCs can each have 1 vertex. This leaves $n - (k - 1) = n - k + 1$ vertices for the largest SCC.

- (c) (2 points) A document consists of symbols A, B, C, D, E with frequencies $2, 2, 2^2, 2^3, 2^4$ respectively.

When applying Huffman encoding to these letters, E requires bit(s).

Solution: 1 bit. Running the algorithm from lecture would give us the encoding of A, B, C, D, E as 1111, 1110, 110, 10, 0, respectively.

- (d) (2 points) If a connected graph has multiple MSTs, every MST uses exactly the same number of edges of each weight. (For example, if one MST uses three edges of weight 5, then every MST uses exactly three edges of weight 5.)

☐ True ☐ False

Solution: True. <https://cs.stackexchange.com/questions/2204/do-the-minimum-spanning-trees-of-a-weighted-graph-have-the-same-number-of-edges>

- (e) (6 points) Consider a DAG on vertices $\{1, 2, \dots, n\}$.

- (i) (2 points) How many distinct simple DAGs have *exactly one* topological order, namely $1, 2, \dots, n$? Express your answer as a function of n . *Hint: Your answer may be expressed as a power of 2.*

Solution: $2^{\frac{(n-1)(n-2)}{2}}$. All $n - 1$ edges of the form $(i, i + 1)$ are required to force uniqueness. The remaining $\binom{n}{2} - (n - 1) = \frac{(n-1)(n-2)}{2}$ directed edges are optional.

- (ii) (2 points) Draw an example (with $n = 4$) of a DAG which has *exactly two* topological orders, one of which is $1, 2, 3, 4$. Label the vertices and edges clearly.

Solution: Example: vertices $\{1, 2, 3, 4\}$ with directed edges

$(1, 2), (1, 3), (2, 4), (3, 4).$

- (iii) (2 points) **Challenging:** For $n \geq 3$, how many distinct simple DAGs have *exactly two* topological orders, one of which is $1, 2, \dots, n$? Give your answer as a function of n .

Solution: $(n+1)2^{\frac{n^2-3n-2}{2}}$. Since there is more than one topological ordering, a directed edge of the form $(k, k+1)$ must not appear. This yields the second topological ordering $1, 2, \dots, k-1, k+1, k, k+2, \dots, n$. All adjacent edges in the second topological ordering except $(k+1, k)$ must appear. All other edges are optional. Now, the number of enforced edges depends on the value of k according to the following two cases:

Boundary cases ($k = 1$ or $k = n - 1$): $(n - 1) + 1 = n$ enforced edges.

Interior cases ($2 \leq k \leq n - 2$): $(n - 1) + 2 = n + 1$ enforced edges.

Thus, the total number of distinct DAGs with exactly two topological orderings is:

$$2 \cdot 2^{\binom{n}{2} - n} + (n - 3) \cdot 2^{\binom{n}{2} - (n+1)} = (n + 1)2^{\frac{n^2-3n-2}{2}}.$$

- (f) (2 points) Consider the following algorithm to construct the maximal spanning tree: Negate each edge and run Kruskal's algorithm. The algorithm always correctly returns the maximal spanning tree.

☐ True ☐ False

Solution: True. By negating each edge weight, what is minimum now becomes a maximum. Furthermore, Kruskal's algorithm does not require any assumptions on the input graph.

- (g) (2 points) Recall that a simple path is a path with no repeated vertices. Consider the following procedure to construct the longest simple path from s to t : negate each edge, then run Bellman-Ford, the algorithm for shortest paths in graphs with negative edges. This procedure always correctly returns the longest simple path from s to t .

☐ True ☐ False

Solution: False. Bellman-Ford may report that a negative cycle exists. Even if we modify Bellman-Ford to skip this check, the internal distance array values do not necessarily correspond to actual simple paths in the graph.

- (h) (4 points) Consider a graph $G = (V, E)$ with a unique shortest path from source s to target t .
- (i) (2 points) Let G contain potentially negative edge weights. Consider the following fix: increase each edge weight by $|\min_{(u,v) \in E} w(u, v)|$, so that all weights become nonnegative.

Running Dijkstra's algorithm on this modified graph will always yield the same shortest path on the original graph.

☐ True ☐ False

Solution: False. While multiplying preserves shortest paths, adding a scalar does not.

- (ii) (2 points) Let G contain only negative edge weights. Consider the following fix: multiply each edge weight by -1 , so that all weights become nonnegative. Running Dijkstra's algorithm on this modified graph will always yield the same shortest path on the original graph.

☐ True ☐ False

Solution: False. The shortest path in G' is the longest path in G .

3 All about pre and post values (13 points)

Suppose G is a directed graph on 6 vertices, and you are given **only** the following 5 pre and post values for DFS performed on G . Note that you cannot make any assumptions on what order the vertices were visited (i.e., what order neighbors were visited when exploring any vertex).

Vertex i	PRE[i]	POST[i]
u	2	7
v	4	5
w	3	6
x	1	8
y	9	12
z	Unknown	Unknown

- (a) (2 points) True or False: x is an ancestor of w in the DFS tree.

☐ True ☐ False

Solution: True. Since $\text{PRE}[x] < \text{PRE}[w] < \text{POST}[w] < \text{POST}[x]$.

- (b) (2 points) **For this part only**, suppose you also know $(u, v) \in E$. What type of edge is (u, v) ?

☐ Tree ☐ Forward ☐ Cross ☐ Back

Solution: Forward. Firstly, we can determine from the same logic as above that u is an ancestor of v , so it's either a tree or a forward edge. It's not a tree edge because we can tell from $(\text{PRE}[w], \text{POST}[w])$ that w is an ancestor of v (as it entirely contains $(\text{PRE}[v], \text{POST}[v])$). The answer is thus Forward.

- (c) (2 points) Fill in the missing pre and post values for the last vertex z :

PRE[z]: POST[z]:

Solution: 10, 11. Since the pre and post values are pairwise distinct and cover all the numbers in $\{1, 2, \dots, 12\}$, we can therefore deduce that $\text{PRE}[z] = 10$, $\text{POST}[z] = 11$ as those are the only two missing numbers.

- (d) (4 points) In the box below, list all directed edges that **must** exist in G , based solely on the given pre and post values.

Solution: $(x, u), (u, w), (w, v), (y, z)$. Based on the pre and post values, we can conclude that there exists a directed path $x \rightarrow u \rightarrow w \rightarrow v$ (since the pre and post values account for the entire interval $[1, 8]$). Given the answer for part (c), we can also tell that there must be an edge (y, z) . Hence, the answer is $(x, u), (u, w), (w, v), (y, z)$.

- (e) (3 points) True or False: There exists an **undirected** graph G' such that $\{z, x\} \in E$ and the given pre and post values are valid for some DFS on G' .

☐ True

☐ False

Solution: False. We can tell that z and x must be in different connected components of G' , and this contradicts the assumption that $\{z, x\} \in E$. Another way to think about it is in a directed graph, the pre and post values indicate that (z, x) is a cross edge in the DFS tree. However, for undirected graphs, cross edges do not appear in their DFS trees.

4 Melting icebergs (24 points)

The penguins of PNP are in danger! As rising temperatures are causing their iceberg island homes to melt, each penguin must try to slide to safety. There are k penguins in total, each starting from their unique iceberg island home.

- (a) (12 points) Let us model the situation as a directed graph $G = (V, E)$, where each vertex $v \in V$ represents an iceberg. If there exists a directed edge $(u, v) \in E$, then a penguin can slide from iceberg u to iceberg v .

An iceberg is considered *safe* if it has no outgoing edges (i.e., penguins that reach it cannot slide anywhere else). A penguin survives if it can eventually reach a safe iceberg.

Design an algorithm to determine if there is at least one penguin that can survive. Your algorithm should have a runtime of $O(|V| + |E|)$. **Briefly justify the runtime of your algorithm.**

Formal Description

Input:

- (a) A directed graph $G = (V, E)$.
- (b) A subset $P \subseteq V$ of size $|P| = k$ consisting of the starting nodes of each penguin.

Output: Whether or not there exists a penguin that can make it to a safe iceberg.

Correct algorithms with runtime $O(k(|V| + |E|))$ will receive partial credit.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Solution: Modify graph G with a dummy source node s as follows: for every starting node $p \in P$, add a directed edge $s \rightarrow p$. Let G' be the resulting modified graph. Now, run DFS/BFS on G' starting at the vertex s . If we eventually reach a vertex with no outgoing edges, return "Yes". Otherwise, return "No".

Runtime Analysis: Creating G' adds at most $|S|$ additional edges and running DFS/BFS takes $O(|V| + |E| + |S|)$. Since $|S| = O(|V|)$, the overall runtime is therefore $O(|V| + |E|)$.

- (b) (12 points) A few of the penguins survived! The survivors now inhabit a subset $S \subseteq V$ of the original icebergs. However, all of the routes between the icebergs melted away, so the penguins want to build two-way bridges between icebergs such that they can travel from any iceberg to any other iceberg. Define the set of all unordered pairs of icebergs to be $\text{Pairs}(S) = \{\{u, v\} \mid u, v \in S\}$.

To ensure safety and order in this iceberg bridge construction project, the PNP safety council has imposed two restrictions:

- For pairs of icebergs $\{u, v\} \in M \subseteq \text{Pairs}(S)$, a bridge $\{u, v\}$ *must* be built.
- For pairs of icebergs $\{a, b\} \in F \subseteq \text{Pairs}(S)$, a bridge $\{a, b\}$ is *forbidden* from being built.

Furthermore, for each potential bridge $\{u, v\}$, building it incurs a cost $c(\{u, v\})$. Design an algorithm to return the minimum total cost of building all the necessary bridges. Your algorithm can return -1 if it is impossible to satisfy the constraints. (No proof or runtime analysis needed.)

Formal Description

Input:

- A set of vertices S .
- A cost $c(\{u, v\})$ for each possible bridge $\{u, v\} \in \text{Pairs}(S)$.
- A set of mandatory bridges $M \subseteq \text{Pairs}(S)$.
- A set of forbidden bridges $F \subseteq \text{Pairs}(S)$.

Output: The minimum total cost of building bridges such that any iceberg $v \in S$ can reach any other iceberg $v' \in S$, obeying the constraints that mandatory bridges must be built, and forbidden bridges must not be built. Return -1 if impossible.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Solution: Create a modified graph G' with vertices S and edges $c(\{u, v\})$. Set the edges in F to ∞ and run Kruskal's algorithm starting with all edges in M to get the MST $T = (V, E)$. If the sum of the costs of the remaining edges in T is ∞ , return -1. Else, return the sum of the costs of the union of the MST's edges, E and M .

Alternate Solution:

Create a modified graph G' with vertices S , edges $c(\{u, v\})$, and set the edges in F to ∞ . Contract each connected component consisting of mandatory bridges M into a single vertex. Run Kruskal's or Prim's algorithm on this modified graph, G' to get the MST $T = (V, E)$. Return -1 if the sum of the costs of the edges in T is at least ∞ . Else, return the sum of the costs of the union of E and M .

5 Playing with polynomials (20 points)

- (a) (5 points) Consider two polynomials, $A(x) = a_0 + a_1x$ and $B(x) = b_0 + b_1x$. The product $A(x)B(x) = a_0b_0 + (a_1b_0 + a_0b_1)x + a_1b_1x^2$ can be *naively* computed by performing four multiplications: a_0b_0 , a_1b_0 , a_0b_1 , and a_1b_1 .

Alternatively, $a_0b_0 + (a_1b_0 + a_0b_1)x + a_1b_1x^2$ can be computed using only three multiplications. List the three products in terms of a_0 , a_1 , b_0 , and b_1 .

Solution: a_0b_0 , a_1b_1 , and $(a_0 + a_1)(b_0 + b_1)$. This is because $a_1b_0 + a_0b_1 = (a_0 + a_1)(b_0 + b_1) - a_0b_0 - a_1b_1$. Students should recall this relation from Lecture 2, as the same trick is used to compute integer multiplication.

- (b) (11 points) Design an efficient algorithm to compute the product of the polynomials $A(x)$ and $B(x)$, both of degree n . You may assume for simplicity that n is of the form $2^i - 1$ for some positive integer i . Your algorithm should have a runtime of $O(n^{\log_2 3})$. (No proof needed.)

Formal Description

Input: $A(x)$ and $B(x)$, specified by their coefficients, $a_0, \dots, a_n$ and $b_0, \dots, b_n$, respectively.

Output: $A(x)B(x)$, specified by its coefficients.

- (c) (4 points) Analyze the runtime of your algorithm.

Solution: In the previous part, notice that for an instance of size n , we recursively called the same algorithm with instance size $n/2$ three times. Moreover, the algorithm ran for $O(n)$ additional time to perform the additions and subtractions of the polynomial coefficients. Thus, if $T(n)$ denotes the runtime of our algorithm for an instance of size n , then we can infer the recurrence relation

$$T(n) = 3T(n/2) + O(n).$$

Using the Master Theorem, we therefore find that $T(n) = O(n^{\log_2 3})$.

Solution: See the solution to discussion 2, problem 2.

Solution: Alternative solution (Kadane's algorithm, dynamic programming). Keep a DP table of length n . The subproblem is defined as the the max subarray sum ending at index i . The update rule is: $DP[i] = \max(DP[i - 1] + A[i], A[i])$. The solution is then $\max DP[i]$ or 0 if everything is negative. Since DP is out of scope/not yet taught in the lectures, no partial credit was given for incorrect attempts.

- (b) (4 points) Analyze the runtime of your algorithm.

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Solution: See the solution to discussion 2, problem 2. $O(n)$ for using Kadane's algorithm.

- (c) (12 points) Let A be a length- n array of two-dimensional vectors of the form $(x, 0)$ or $(0, y)$. We denote the i 'th element of the array to be $(A[i, 1], A[i, 2])$ where either $A[i, 1]$ or $A[i, 2]$ is 0 (or possibly both).

The *size* of an empty array is 0. For $1 \leq i \leq j \leq n + 1$, the *size* of the corresponding non-empty subarray is given by

$$\text{size}(A, i, j) = \left| \sum_{k=i}^{j-1} A[k, 1] \right| + \left| \sum_{k=i}^{j-1} A[k, 2] \right|$$

As an example, consider $A = [(0, -5), (0, 4), (3, 0), (0, 2), (-7, 0), (1, 0)]$. The maximum size of A is $|3 + (-7)| + |4 + 2| = 10$ and is given by the contiguous subarray $[(0, 4), (3, 0), (0, 2), (-7, 0)]$.

Design an efficient algorithm that finds the maximum subarray size, $\max_{1 \leq i \leq j \leq n+1} \text{size}(A, i, j)$.

Formal Description

Input: An array A of n two-dimensional vectors. For any vector $(A[k, 1], A[k, 2])$ in A , $A[k, 1] = 0$ or $A[k, 2] = 0$, or both.

Output: $\max_{1 \leq i \leq j \leq n+1} \text{size}(A, i, j)$, with the size function defined above.

Answer box on next page.

This image shows a single sheet of white paper with horizontal ruling lines. The lines are evenly spaced and run across the width of the page. There is no text or other markings on the paper.

Solution: Divide the array into two halves, recurse on each half, and get d_1 from the left half and d_2 from the right.

The base case is when the array is of length 1, which should simply return the Manhattan distance of that single vector.

In the conquer phase, sweep from the middle to the left, accumulating the vector sum (x, y) and tracking $L_1 = \max(x + y)$, $L_2 = \max(x - y)$, $L_3 = \max(-x + y)$, and $L_4 = \max(-x - y)$. Similarly, sweep from the middle to the right and keep track of R_1, R_2, R_3 , and R_4 . Then, the max distance that crosses the border is $d_3 = \max(L_1 + R_1, L_2 + R_2, L_3 + R_3, L_4 + R_4)$. Return $\max(d_1, d_2, d_3)$.

(d) (3 points) The runtime of your algorithm is .

Solution: $O(n \log n)$. The merge phase takes a single sweep to get all the values we need, which is $O(n)$. Therefore, the recurrence relation for the algorithm is $T(n) = 2T(n) + O(n)$, which gives $O(n \log n)$

Solution:

- (a) Sort all sprinklers by the right endpoint r_i in non-decreasing order. Let the resulting ordering be $[l_{\pi(1)}, r_{\pi(1)}], [l_{\pi(2)}, r_{\pi(2)}], \dots, [l_{\pi(n)}, r_{\pi(n)}]$.
- (b) Maintain a set P of sprinkler positions. Initialize it to $P = \emptyset$.
- (c) For $j = 1, 2, \dots, n$, do the following:
 - If there is no sprinkler in P that is in the interval $[l_{\pi(j)}, r_{\pi(j)}]$, add $r_{\pi(j)}$ to P .
- (d) Return $|P|$ and P .

8 Sasha's party (30 points)

It's Sasha's birthday! Her k friends are traveling by bus to the party, each starting from a bus stop. Unfortunately, Sasha's friends forgot to buy presents, so each friend must stop at a gift shop on their way to the party. Luckily, there is a subset S of bus stops that are located next to gift shops.

- (a) (15 points) Design an efficient algorithm that returns the shortest amount of time for Sasha's friends to *all* reach the party location, while each passing through **at least one gift shop**. For full credit, your runtime must not depend on k or $|S|$. (No proof or runtime analysis needed.)

Formal Description

Input:

- (a) A directed graph $G = (V, E)$, where each vertex $v \in V$ represents a bus stop. It is possible to travel from u to v if the directed edge (u, v) is in E .
- (b) Positive integer edge weights $t(u, v)$ representing the travel time from u to v .
- (c) A subset $F \subseteq V$ of size k representing the initial locations of Sasha's friends.
- (d) A subset $S \subseteq V$ representing bus stops with gift shops.
- (e) A vertex $p \in V$ representing the party location.

Output: The shortest amount of time for Sasha’s friends to all arrive at p , given that each one has to pass through at least one gift shop.

This image shows a full page of white paper with horizontal dotted lines. The lines are evenly spaced and run across the entire width of the page, providing a guide for handwriting practice. There are no margins, text, or other markings on the paper.

Solution: Create top copies $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$ of the original graph G but with their edges reversed. Concretely, set $V_1 = V_2 = V$ and $E_1 = E_2 = \{(v, u) : (u, v) \in E\}$.

Combine the two ‘reversed’ copies G_1 and G_2 into a larger graph G' as follows: create edges of weight 0 from each gift shop in G_2 to its corresponding gift shop in G_1 . Run Dijkstra on G' starting from p in G_2 and return the value $\max(\text{distance to every } f \in F \text{ in } G_1)$.

Reversing the edges allows us to run Dijkstra only once at the party instead of running Dijkstra k times for each friend. Creating edges from the gift shops from G_2 and G_1 forces our path to any $f \in F$ in G_1 to have visited at least one gift shop.

- (b) (15 points) Sasha is heading home from her party, but some of the buses have ended their operation for the day. For each pair of bus stops (u, v) , Sasha knows whether she can take the bus from u to v , or if she will need to walk from u to v . She also knows the travel time from u to v . There is **only one** possible mode of transportation (bus or walk) from u to v .

Design an efficient algorithm that returns the path from the party to Sasha’s home that minimizes her total travel time. If multiple paths achieve the minimal travel time, return the path that minimizes the number of times she switches between transportation modes. Sasha’s initial transportation mode at the party, before beginning to traverse between bus stops, is **walking**.

Formal Description

Input:

- (a) A simple directed graph $G = (V, E)$, where each vertex $v \in V$ represents a bus stop. It is possible to travel from u to v if the directed edge (u, v) is in E .
- (b) Positive integer edge weights $t(u, v)$ representing the travel time from u to v .
- (c) Edge labels $m(u, v) \in \{\text{bus}, \text{walk}\}$ representing the transportation mode from u to v .
- (d) A vertex $p \in V$ representing the party location. Prior to traversing an edge from p , Sasha is **walking**.
- (e) A vertex $d \in V$ representing Sasha’s home.

Output: A path $(p = v_0, v_1, \dots, v_k = d)$ that minimizes the total travel time, $\sum_{i=0}^{k-1} t(v_i, v_{i+1})$. Among all paths that achieve the minimal travel time, minimize the number of switches between transportation modes, $\mathbf{1}\{\text{walk} \neq m(v_0, v_1)\} + \sum_{i=0}^{k-2} \mathbf{1}\{m(v_i, v_{i+1}) \neq m(v_{i+1}, v_{i+2})\}$.

Answer box on next page.

First solution: We can solve this problem by modifying Dijkstra. In addition to storing the distance to each vertex in the priority queue, we will also store the number of transport method switches that it takes to reach that vertex. Since Sasha started off walking, if she has done an even number of transport switches at any point, then that means her current mode of transport is walking; otherwise, her current mode of transport is taking the bus.

```

if  $dist(v) < dist(u) + w(u, v)$  :
     $dist(v) = dist(u) + w(u, v)$ 
if  $dist(v) == dist(u) + w(u, v)$  :
    if  $s(u) \% 2 == 0$  :
        if  $m(u, v)$  is walk:
             $s(v) = s(u)$ 
        else:
             $s(v) = s(u) + 1$ 
    else if  $s(u) \% 2 == 1$  :
        if  $m(u, v)$  is bus:
             $s(v) = s(u)$ 
        else:
             $s(v) = s(u) + 1$ 

```

Proof of correctness: Observe that our extra tracking of s does not affect the distance function at all for each vertex — the heap only uses s to break ties, and we only do the extra work when the distances are equal, and during the extra work, we do not reassign $dist$ values. Therefore, we can safely infer from Dijkstra's algorithm that the distance outputted is the minimum possible.

Second solution: Construct two copies G_w and G_b of the original graph G , one denoting the state of walking and the other denoting the state of taking the bus, respectively. Set the edge weights in the graphs G_w and G_b to be the same as those in G but multiplied by $n + 1$. Now, construct a bigger graph G' consisting of G_w and G_b and bi-directional edges of weight exactly 1 between each vertex v in G_w and its corresponding copy in G_b . Run Dijkstra from p in G_w and return the shorter distance from p in G_w to either d in G_w or d in G_b .

Proof of correctness: Since any path in G would switch the mode of transportation at most n times, observe that any path in G' would switch between G_w and G_b at most n times. Since we multiplied the weights of G by $n + 1$ and are all integers, then we therefore deduce that for two paths P_1 and P_2 in G' , P_1 is shorter than P_2 if and only if either the corresponding path of P_1 in G is shorter than the corresponding path of P_2 in G , or the corresponding paths of P_1 and P_2 in G have the same length, but P_1 used fewer modes of transportation. Thus, the shortest path in G' from p in G_w to either copy of d will be the shortest path in G from p to d with the fewest switches in modes of transportation.

Assumptions

(0 points) Congratulations! You're done with the exam. Were there any ambiguities in the exam? If so, record the ambiguity and the assumptions that you made in the box below, as well as possible alternate interpretations and their associated answers.

For example, "If question X is asking about A, then my answer is Y. But, if the question is about B, then my answer is Z." If you submit a regrade request regarding any ambiguities on the exam, we will only consider the request if you have correctly noted an ambiguity in the box below.

Comments, Concerns, etc.

(0 points) Did you notice anything suspicious during the exam? Please note in the box below. Where were you sitting, and what did you notice?

(0 points) Any other comments, concerns, qualms, queries, or suggestions?

(0 points) 100 penguins versus one CS 170 student. Who would win? Justify your answer.

Use this page for scratch work.